

Architectural Reincarnation: The Cyclical Evolution of Virtual Machines, Component Frameworks, and Context Protocols

The history of software architecture is frequently misinterpreted as a linear trajectory of continuous obsolescence, wherein outdated paradigms are permanently discarded in favor of novel innovations. A more rigorous, longitudinal analysis of computer science reveals a vastly different reality: software architecture is a cyclical discipline. Paradigms such as virtual machines, remote procedure calls (RPC), and component-based software rarely vanish entirely. Instead, they recede when constrained by the hardware limitations, network latency, or engineering cultural realities of their specific epochs, only to resurface decades later in adapted, highly scalable forms. Technologies once deemed "lost" or commercially defeated—such as the UCSD p-System, Apple's OpenDoc, IBM's System Object Model (SOM), and early binary RPC frameworks like DCE-RPC—established foundational abstractions that continue to implicitly govern modern computing.

As the technology sector transitions into an era dominated by artificial intelligence, autonomous agents, and large language models (LLMs), historical challenges surrounding system interoperability, context preservation, and application malleability have returned to the absolute forefront of system design. Modern frameworks like WebAssembly (Wasm) and Anthropic's Model Context Protocol (MCP) are not unprecedented leaps; they are the direct, conceptual descendants of these historical architectures, solving identical fundamental problems of computation and communication at vastly different scales. This comprehensive analysis explores the technical architectures, historical demises, and modern reincarnations of these foundational technologies, demonstrating how the persistent tension between technical elegance and market pragmatism continues to drive the evolution of software.

The Abstraction of Execution: Virtual Machines and Bytecode

The conceptual desire for "write once, run anywhere" portability long predates the widespread adoption of the Java Virtual Machine (JVM) in the mid-1990s. In the highly fragmented microcomputer and minicomputer markets of the 1970s, establishing a uniform execution environment across fundamentally disparate hardware architectures was a critical, existential challenge for software portability.

The UCSD p-System: The Genesis of the Portable Virtual Machine

Developed at the University of California, San Diego (UCSD) in 1977 under the direction of

Kenneth Bowles, the UCSD p-System stands as one of the earliest and most successful implementations of a portable bytecode interpreter and hardware-agnostic operating system.¹ Designed primarily to support the academic instruction and widespread deployment of the Pascal programming language, the system deliberately abstracted away the underlying hardware by compiling source code into an intermediate "pseudo-code" (p-code) rather than native machine language.² This p-code was then executed by a virtual processor, known natively as the p-Machine.³

The p-Machine was designed specifically as a stack machine, a radical architectural departure from the register-centric architectures (such as Complex Instruction Set Computing or Reduced Instruction Set Computing) that dominate physical CPU designs.⁴ While modern physical processors rely on numerous general-purpose registers to accelerate operations—often requiring complex internal logic to manage register-to-register state—the p-Machine featured zero general-purpose registers.⁴ Instead, it utilized exactly eight specific registers that acted as nominal 16-bit pointers to physical locations in main memory.⁴ These included:

1. **SP (Stack Pointer):** Tracking the current execution stack.
2. **IPC (Interpreter Program Counter):** Pointing to the next instruction byte to execute.
3. **SEG:** Pointing to the Procedure Dictionary of the segment to which the currently executing procedure belongs.
4. **JTAB:** Pointing to the current segment's table of attributes and jump table offsets.
5. **KP:** The base of the stack, growing downward toward the heap.
6. **MP (MarkStack Pointer):** Pointing into the current activation record where local variables are isolated.
7. **NP (New Pointer):** Pointing to the top of the heap, growing upward from high memory.
8. **BASE:** Pointing to the activation record of the most recently invoked base procedure (level 0), which facilitated global variable access.⁴

This stack-based architecture was actively chosen for two critical reasons: implementation simplicity and binary code compactness.⁴ Because the virtual machine interpreter had to be written entirely in the native assembly language of every target hardware platform—ranging from the Apple II and IBM PC to the DEC PDP-11 and Zilog Z80—minimizing the complexity of the interpreter was essential.¹ Furthermore, because operations inherently drew their operands from the stack and returned computed results directly to the stack, instructions did not need to explicitly encode register addresses or complex addressing modes. Consequently, all core opcodes in the p-Machine were a single byte in length.⁴ For example, the SLDC (Short Load Constant) instruction utilized half of the entire available opcode space to load integer constants from 0 to 127 onto the stack in a single byte.⁴ Another specialized opcode, LSA (Load String Address), pushed the address of a Pascal-style string directly onto the stack and automatically advanced the IPC past the length of the string characters in the instruction stream.⁴ This resulted in extremely compact binaries that were historically vital for systems possessing minimal RAM and floppy disk storage capacities.⁴

The architectural legacy of the p-System is profound. To bypass severe RAM constraints, the

p-System utilized a segment directory structure where code files were divided into a maximum of 16 segments.⁴ The first 512-byte block functioned as a directory, and segments could be demand-loaded by the operating system directly into memory.⁴ This demand-paged virtual machine model served as the direct conceptual precursor to the dynamic class loading mechanisms found in modern bytecode environments.¹ Despite dying a slow death after its final Version IV release in 1987, the abstraction provided by the p-Machine fundamentally proved that software could be entirely decoupled from silicon.⁷

The JVM and WebAssembly: Scaling the Stack Machine

While the p-System faded as hardware standardization (driven almost entirely by the x86 architecture and MS-DOS) rendered its primary selling point of portability less critical for consumer software, the virtual machine concept aggressively resurfaced in the mid-1990s with the Java Virtual Machine (JVM). Like the p-System, the JVM is a stack-based virtual machine designed to meticulously hide underlying operating system and hardware differences.¹ Both systems share the fundamental philosophy of utilizing programs written to a virtualized instruction set to provide cross-platform support.¹ However, the JVM was structurally optimized for large-scale enterprise applications, utilizing a rigid ClassFile structure containing rich metadata, constants, method definitions, fields, and deep inheritance hierarchies.⁹ Today, the culmination of this bytecode lineage is WebAssembly (Wasm). Wasm fundamentally shares the "write once, run anywhere" ethos of the p-System and the JVM, but strategically extends it to the web browser and modern edge computing environments.¹⁰ Like its predecessors, Wasm utilizes a structured stack machine model, operating on a linear memory space.⁸ However, it diverges significantly from the JVM in its foundational intent and security posture. While the JVM was initially designed exclusively to support Java and its specific object-oriented family of languages (eventually accommodating Scala and Kotlin), Wasm was designed from its inception to act as a universal, language-agnostic compilation target.⁸ Furthermore, Wasm employs a strict "deny-by-default" sandboxing model.¹⁰ Because it was born in the browser—an environment where visiting a website necessarily involves downloading and executing highly untrusted code—Wasm isolates modules entirely, denying access to system resources, the host filesystem, or network sockets unless explicitly authorized by the host environment.¹⁰ Wasm structures its code into modules encompassing types, functions, and memory, firmly rejecting the strictly object-oriented ClassFile paradigm of the JVM.⁹ In standalone runtimes such as wasmtime, the architecture revolves around an Engine representing the global compilation context, and a Store which maintains isolated context for WASM objects within a specific thread.⁹

The evolution from p-code to WebAssembly demonstrates that while hardware eventually caught up to software ambitions, the fundamental requirement to isolate software logic from the underlying execution environment remains a permanent fixture of computer science.

The Network as the Computer: The Evolution of

Remote Procedure Calls

Just as virtual machines abstracted the local processor, Remote Procedure Call (RPC) systems emerged to abstract the network. The overarching goal of RPC architecture is to allow a local client process to invoke a procedure or function on a remote server with the exact same semantics as a local function call, systematically hiding the immense complexities of network transport, socket management, latency, and data marshaling.¹²

Canonical vs. Multi-Canonical Data Representation

One of the most persistent and computationally expensive challenges in distributed computing is the disparity in data representation across heterogeneous hardware architectures. Different microprocessors handle byte ordering (big-endian versus little-endian), integer sizing, and floating-point representations in fundamentally incompatible ways. The early binary RPC systems approached this interoperability problem with two opposing, highly illustrative philosophies.

Sun Microsystems' Open Network Computing RPC (ONC RPC), the foundational protocol supporting the Network File System (NFS), pioneered the use of a strict canonical format.¹⁴ ONC RPC defined the eXternal Data Representation (XDR) standard.¹² Under the XDR model, all data must be forcefully converted by the sender into a standard, universally agreed-upon binary format before network transmission.¹² The receiver then decodes the XDR format into its own local representation.¹² While mathematically elegant and simple to reason about, this "canonical" approach introduces a significant and unavoidable performance penalty: it forces computationally expensive data conversions on every single message transmission, even when the client and the server happen to share the exact same hardware architecture.¹⁶

Conversely, the Distributed Computing Environment RPC (DCE RPC), developed by the Open Software Foundation, introduced the Network Data Representation (NDR) protocol, which utilizes a highly optimized "receiver-makes-right" or multi-canonical philosophy.¹² In DCE RPC, the sender transmits data natively in its own format. It achieves interoperability by appending a 4-byte format label to the Protocol Data Unit (PDU) header that explicitly identifies the sender's character encoding (e.g., ASCII or EBCDIC), integer representation, and floating-point format.¹⁷ The receiver inspects this label upon packet arrival; if the sender's format strictly matches the receiver's local format, the data is passed into memory and processed directly with zero conversion overhead.¹⁶ Conversion is only performed by the receiver when an architectural mismatch occurs.¹⁶

DCE RPC also introduced advanced directory services and Universally Unique Identifiers (UUIDs) generated by `uuidgen` for service identification, actively bypassing the arbitrary, programmer-defined integer program numbers that ONC RPC relied upon to identify servers.¹⁶ In DCE, an Interface Definition Language (IDL) compiler generated C headers and client/server stubs to map complex incoming parameters into localized memory structures.²⁰

From Binary RPC to HTTP Protocols and Web Services

As software engineering underwent a massive paradigm shift toward object-oriented programming in the 1990s, binary RPC frameworks rapidly evolved to manage distributed objects and state across networks. This evolution gave rise to Microsoft's Distributed Component Object Model (DCOM), Java's Remote Method Invocation (RMI), and the Common Object Request Broker Architecture (CORBA).¹⁵ Java RMI, for instance, modernized lifecycle management by utilizing dynamic stub generation and a lease-based distributed garbage collection protocol to track the lifecycle of remote objects.¹⁵

However, as the World Wide Web exploded in popularity, these dense binary protocols proved exceptionally fragile over the open internet. They suffered from severe firewall restrictions, intense binary coupling, and a lack of human readability. Consequently, the industry pivoted aggressively toward text-based serialization formats operating over standard HTTP networks.¹⁶ XML-RPC emerged as a simplistic, early web service protocol that marshaled data into XML messages.¹⁶ It rapidly ballooning into the highly complex, enterprise-heavy Simple Object Access Protocol (SOAP).¹⁶ SOAP extended XML-RPC by adding support for user-defined types and complex message routing, but its overwhelming verbosity and labyrinthine complexity led to an industry-wide backlash.¹⁶

Ultimately, both were largely displaced by Representational State Transfer (REST) and later JSON-RPC. REST gained ubiquitous, global dominance not because it provided the strict contractual guarantees of DCE RPC, nor the type safety of CORBA. Instead, its stateless, resource-oriented model mapped perfectly to standard HTTP verbs (GET, POST, PUT, DELETE), acting as a natural proxy for CRUD (Create, Read, Update, Delete) operations.¹⁶ REST was built for humans; it was loose, flexible, and placed the burden of understanding data shapes directly on the developer reading documentation.²² Yet, as will be discussed regarding Anthropic's Model Context Protocol (MCP), the architectural pendulum is currently swinging violently back toward the strict schema guarantees and stateful sessions of early RPC models to satisfy the unique needs of non-human consumers like AI agents.²²

The Component Software Dream: Modularity and its Demise

In the early 1990s, the software industry identified a severe architectural flaw in application design: monolithic applications were becoming uncontrollably bloated. Software vendors were engaged in a feature war, forcing users to purchase massive, resource-heavy office suites merely to access a single, highly specific feature. The proposed, utopian solution was component-based software architecture—a paradigm designed to allow users to assemble customized "compound documents" from atomic, infinitely reusable parts developed by disparate vendors.²³

The Rise and Fall of OpenDoc

OpenDoc was a highly ambitious software componentry framework championed by an industry consortium known as the AIM alliance (Apple, IBM, and Motorola), with WordPerfect later joining.²³ It was expressly created as an open-standard alternative to challenge Microsoft's rapidly growing, proprietary Object Linking and Embedding (OLE) standard.²³ The core philosophy of OpenDoc was a radical shift from application-centric computing to document-centric computing.²³ Rather than launching a monolithic word processor, a user would open a blank digital canvas. The user could then drag and drop small, specialized components—called "parts"—onto this canvas.²³ One vendor might supply the text editing part, another a bitmap editing part, and a third a tool for browsing an FTP server.²³ OpenDoc provided the unifying framework in which these disparate components could run seamlessly together, alongside a compound document format for storing the serialized data.²³ Despite its visionary goals and immense corporate backing, OpenDoc was an unequivocal, spectacular failure.²⁶ Its demise was rooted in a fatal confluence of severe hardware constraints, internal corporate political infighting, and crippling architectural complexity.²⁵ From a hardware perspective, OpenDoc was built for an era of computing power that had not yet arrived. In 1996, the average consumer Macintosh possessed approximately 2 megabytes of RAM.²⁵ The OpenDoc framework, however, was incredibly resource-intensive; it required an absolute minimum of 4 megabytes of RAM to execute, and practically demanded 8 megabytes to run multiple parts smoothly.²⁵ This induced intense friction with Apple's own Copland OS development team, who were deeply fearful of OpenDoc's massive memory footprint and actively resisted integrating it into the base operating system.²⁵ Furthermore, OpenDoc suffered from catastrophic "second-system effect" complexity. The OpenDoc human interface team attempted to completely reinvent the Macintosh user experience, changing familiar interface paradigms (such as removing the standard "Quit" command and renaming the "File" menu to "Document") which alienated and intimidated developers.²⁵ Technically, integrating OpenDoc into legacy, non-object-oriented procedural codebases was a nightmare. When management ordered the ClarisWorks team to integrate OpenDoc into their aging 1989 codebase, the friction was so severe that key engineers defected to Microsoft, and others quit entirely to form Gobe Software.²⁵ The framework strictly demanded pure object-oriented C++ architectures and made rigid assumptions about application design.²⁵ Microsoft's OLE, by stark contrast, adopted a much more pragmatic, albeit less technically elegant, approach that was specifically designed to be integrated into existing, legacy software without requiring total application rewrites.²⁵ Additionally, OpenDoc proponents failed to solve the "version-skew" problem.²⁵ If a compound document was built from highly specific component parts provided by vendor A, vendor B, and vendor C, and was subsequently emailed to a colleague who lacked the component from vendor B, the document fractured.²⁵ Without a centralized component economy to distribute these atomic parts, the entire ecosystem collapsed. Upon his return to Apple in 1997, Steve Jobs ruthlessly terminated the OpenDoc project entirely.²³

Feature / Architecture	OpenDoc (Apple/IBM)	OLE / COM (Microsoft)
------------------------	---------------------	-----------------------

Core Paradigm	Document-centric (Parts assembled on a canvas)	Application-centric (Embedding objects within host apps)
Primary Framework	System Object Model (SOM)	Component Object Model (COM)
Legacy Integration	Extremely difficult; required OOP C++ rewrites	Pragmatic; designed for retrofitting existing C code
Interface Mutability	Dynamic method resolution (Flexible)	Immutable interfaces; required new vtables
Primary Cause of Death	Excessive RAM demands, version skew, Steve Jobs' cuts	Deprecated in favor of .NET and web technologies

SOM vs. COM: The Fragile Binary Interface Problem

At the deep architectural foundation of these competing component models lay the object-sharing infrastructure. Microsoft utilized the Component Object Model (COM), while IBM contributed the System Object Model (SOM) to the OpenDoc ecosystem.²⁸ The technological divide between these two models perfectly illustrates the difficulty of building robust, extensible software.

A central issue in object-oriented componentry is the "Fragile Base Class" problem, more accurately termed the Fragile Binary Interface (FBI) problem.²⁹ In older compiled languages like C++, an object's public methods are accessed via a virtual method table (vtable)—an array of function pointers.²⁹ The exact memory offset of each method is hardcoded during compilation. If a library author releases an update that adds a single new virtual method to a base class, it fundamentally alters the memory layout and shifts the vtable offsets.²⁹ Consequently, any derived classes or external third-party client applications compiled against the older binary interface will instantly crash, requiring an ecosystem-wide recompilation.²⁹

Microsoft's COM avoided the fragile base class problem by fiat: COM interfaces were declared strictly immutable.³¹ Once a COM interface (like IUnknown) was published, it could never, ever be changed. To add functionality, developers were forced to publish entirely new interfaces (e.g., IUnknown2).³¹ Furthermore, COM did not support true implementation inheritance, forcing developers to write extensive boilerplate code to implement COM's semantic rules in every application.²⁸

IBM's SOM took a vastly superior, highly dynamic approach.³³ SOM resolved method offsets at runtime through an algorithm that navigated the inheritance tree, fully decoupling the object's

public interface from its underlying binary implementation.²⁸ This allowed SOM library developers to safely issue updates, add methods, and change internal class structures without ever breaking backward compatibility or requiring client software recompilation.³³ Furthermore, while COM only managed interfaces, SOM natively supported full implementation inheritance—and brilliantly, it supported inheritance across completely different programming languages.²⁸ Despite SOM's overwhelming technical superiority and elegance, it suffered the exact same fate as OpenDoc, largely outmaneuvered by Microsoft's aggressive bundling of COM and OLE directly into the Windows operating system.

ActiveX, Bonobo, and Modern Component Reincarnations

Following the defeat of OpenDoc, Microsoft rebranded its OLE controls as ActiveX—a technology designed to run interactive components inside web browsers and Windows applications.²⁵ ActiveX gained immense enterprise traction but proved to be a catastrophic security vulnerability.²⁵ Creating ActiveX controls required mastering immense complexity (often implementing up to six core COM interfaces), leading to severe platform fragmentation.²⁵ More critically, ActiveX controls ran directly inside the browser with full system permissions, completely lacking the vital sandboxing that characterized the Java Virtual Machine.²⁵ Relying entirely on a flawed "Authenticode" code-signing trust model, ActiveX became a premier vector for global malware before being ultimately rendered obsolete by native HTML5 capabilities.²⁵

In the open-source Linux ecosystem, the GNOME desktop environment attempted to rigorously replicate the OLE and OpenDoc component vision through a framework called Bonobo.²⁵ Bonobo utilized CORBA to allow applications like Nautilus to componentize web rendering and audio playback.²⁵ However, the rigorous architectural discipline, immense upfront design effort, and extensive documentation required to build componentized software deeply clashed with the decentralized, "scratch your own itch" culture of open-source engineering.²⁵ User testing of Bonobo environments revealed confusing, fragile interfaces with heavily duplicated functionality (e.g., shipping five distinct, incompatible clock applets), leading to its eventual abandonment in favor of simpler message-passing mechanisms like D-Bus.²⁵ Today, the spirit of OpenDoc survives predominantly in Web Components—a suite of modern browser standards that allow developers to create encapsulated, reusable HTML tags.²⁵ Yet, the true realization of the component software dream is currently emerging not in graphical user interfaces, but in the unseen realm of artificial intelligence agents.

The Philosophy of Pragmatism: "Worse is Better"

The recurring historical defeat of highly advanced, theoretically pure architectures (like OpenDoc, SOM, and CORBA) by purportedly inferior, less elegant technologies (like OLE, COM, and REST) cannot be attributed merely to marketing. It is best understood through the "Worse is Better" philosophy, a term famously coined by computer scientist Richard P. Gabriel in his 1989 essay, "Lisp: Good News, Bad News, How to Win Big".³⁴

Gabriel identified a foundational split in software design. The "MIT Approach" (or "The Right

Thing") dictates that software must be perfectly correct, consistent, and complete, prioritizing *interface simplicity* (ease of use for the consumer) over *implementation simplicity* (ease of building the system).³⁴ OpenDoc and SOM were quintessential "Right Thing" technologies. They attempted to solve monumentally difficult computer science problems—such as cross-platform version skew, dynamic method resolution, and compound document modality—and completely hid that immense complexity from the end user.³⁵ However, achieving this required massive engineering overhead, resulting in heavily delayed, bloated, and unstable releases.²⁵

Conversely, the "Worse is Better" (or "New Jersey style") model argues that *implementation simplicity* possesses vastly superior evolutionary survival characteristics.³⁴ "Worse" technologies actively leak complexity to the user or the developer, but because they are vastly easier to build, they make it to market exponentially faster.³⁵ Gabriel famously used this to explain why the hacked-together C programming language and Unix outcompeted the theoretically superior Lisp machines.³⁴

This philosophy perfectly explains the triumph of REST over DCE-RPC and CORBA. REST offers none of the strict binary typing, efficiency, or state guarantees of advanced RPCs.²² It simply maps loose data strings to HTTP verbs, forcing the human developer to handle missing schemas, parse errors, and broken API contracts.²² REST won the web precisely because its implementation simplicity allowed rapid, viral adoption.²² Evolutionary superiority—shipping an 80% solution rapidly at half the cost—consistently trumps technical completeness in the marketplace.³⁶

The Convergence: AI Agents and the Model Context Protocol (MCP)

For over two decades, REST has remained the dominant API paradigm. However, the explosive rise of Large Language Models (LLMs) and autonomous AI agents has abruptly exposed the severe limitations of the "Worse is Better" REST architecture when applied to non-human actors. REST was meticulously designed for human engineers.²² Humans possess the cognitive flexibility to read poorly written API documentation, adapt to disparate authentication schemas, and infer complex data structures.²² For an AI model, however, a REST API is essentially unstructured noise.²² An LLM encountering a generic GET /users/123 endpoint has no inherent mechanism to know the schema of the response, the required headers, the error codes, or whether it can filter or paginate the data.²² To connect LLMs to existing data sources, developers are currently forced to write custom, brittle prompt engineering wrappers for every single integration.²² The industry suffers from a debilitating $O(N \times M)$ integration problem: connecting N distinct AI models to M heterogeneous APIs requires $N \times M$ custom implementations.³⁸

The Model Context Protocol (MCP), developed and open-sourced by Anthropic in November 2024, resolves this systemic bottleneck by acting as a universal, open standard translation

layer.³⁹ In doing so, MCP effectively resurrects the strict, machine-readable contracts of historical RPCs (like DCE-RPC) alongside the modular, pluggable "parts" philosophy of OpenDoc, tailored specifically for the consumption of artificial intelligence.³⁹

MCP Architecture: A Return to Stateful JSON-RPC

MCP deliberately eschews the stateless nature of REST in favor of JSON-RPC 2.0.⁴² In a modern software landscape utterly obsessed with statelessness, MCP makes the radical, highly effective choice to establish *stateful sessions* between an AI application and an external data server.⁴¹

In the MCP architecture, the AI application (such as an IDE or chat interface) acts as the Host.⁴¹ The Host instantiates isolated, stateful MCP Clients that connect to specific external MCP Servers (which can be local databases, file systems, code repositories, or remote enterprise APIs like Slack or Google Drive).²² These client-server connections are established at the Transport Layer using standard I/O (stdio) for local processes, or Server-Sent Events (SSE) over HTTP for remote servers.²²

Unlike REST, every MCP session begins with a strict protocol and capability negotiation handshake.⁴¹ The client declares its capabilities (e.g., sampling, elicitation, roots) and the server responds with its metadata and supported features.⁴¹ Once the initialization phase is complete and the connection is established, the MCP server exposes three core Data Layer primitives to the AI:

1. **Tools:** Executable software functions (e.g., querying a Postgres database, booking a calendar meeting).³⁹ Crucially, tools declare their precise schema and parameters upfront, allowing the LLM to understand exactly how to invoke them without reading external API documentation.²² The actual execution remains strictly under the Host application's control, ensuring a human-in-the-loop security boundary before any action is executed.⁴¹
2. **Resources:** Contextual data objects (e.g., file contents, application logs) that the AI can read to gain real-world grounding.⁴¹ MCP allows for dynamic updates; if a file changes locally, the server immediately notifies the client via JSON-RPC subscription notifications, ensuring the LLM's context window is never stale.⁴¹
3. **Prompts:** Reusable, server-defined workflow templates provided to the client to streamline and simplify common LLM interactions within that specific domain.⁴¹

By standardizing self-description and wrapping underlying, heterogeneous systems in an MCP server, the AI interacts with a perfectly homogenous protocol regardless of whether the backend is a complex SaaS platform or a local file directory.²² MCP represents the ultimate synthesis of Gabriel's conflicting philosophies: it relies heavily on "worse is better" transport layers (standard JSON-RPC and SSE rather than inventing a complex new binary serialization protocol), while successfully implementing "The Right Thing" at the semantic layer by rigorously enforcing strict, machine-readable contracts.²²

Resurfacing the Past: Malleable Software in the Age of Generative AI

Technologies such as the UCSD p-System, OpenDoc, IBM SOM, and early binary RPC mechanisms are frequently dismissed as mere historical footnotes. However, they were pioneering, visionary efforts to solve the enduring, fundamental challenges of software abstraction: detaching execution from silicon, detaching remote procedures from local memory, and detaching user features from monolithic, siloed applications.

OpenDoc failed because the friction of manually integrating human-coded, memory-intensive C++ parts was insurmountable in the mid-1990s.²⁵ However, today, researchers in fields like "Malleable Software"—such as those at the independent research lab Ink & Switch—are observing that the core vision of OpenDoc is finally becoming technically viable.⁴⁷ The vision of a computing environment uniquely molded by the user from dynamic, interactive parts is returning, utilizing "dynamic documents" like Ink & Switch's *Embark* system, where users gradually enrich text with interactive behavior.⁴⁸

The historical barriers to this reality are collapsing precisely because Artificial Intelligence and LLMs (such as Anthropic's Claude) can now autonomously generate, integrate, and manipulate code components on the fly.⁴⁷ AI serves as the missing, hyper-intelligent compiler for human intent, capable of seamlessly bridging the version-skew and integration problems that historically doomed manual component frameworks.

The Model Context Protocol acts as the vital connective tissue for this rapidly approaching era of composable software. Just as DCE-RPC standardized network procedures, and WebAssembly standardizes highly secure virtual execution, MCP is standardizing the interface between machine reasoning and digital context. The software industry is not inventing entirely new paradigms out of thin air; rather, it is successfully implementing the ambitious, "lost" dreams of the past, leveraging the vast compute, memory, and artificial intelligence capabilities of the present.

Works cited

1. UCSD Pascal - Wikipedia, accessed June 12, 2026, https://en.wikipedia.org/wiki/UCSD_Pascal
2. by NSA on 12-01-2011, Transparency Case# 5335]UNCLASSIFIED The UCSD p-System, accessed June 12, 2026, https://www.nsa.gov/portals/75/documents/news-features/declassified-documents/cryptologic-quarterly/The_UCSD_p_System.pdf
3. SofTech MicroSystems P-System Internal Architecture Reference Manual - Pascal for small machines, accessed June 12, 2026, http://pascal.hansotten.com/uploads/ucsd/softech/SofTech%20MicroSystems_P-System_Internal_Architecture_Reference_Manual.pdf
4. UCSD Pascal In Depth: The p-Machine – markbessey.blog, accessed June 12, 2026,

- <https://markbessey.blog/2025/05/20/ucsd-pascal-in-depth-the-p-machine/>
5. Lecture 3: Instruction Set Architecture - Computer Science - University of California San Diego, accessed June 12, 2026,
https://cseweb.ucsd.edu/classes/fa15/cse30/lectures/PI_CSE30_lecture_3_part2_ARM_after.pdf
 6. A Blast From The Past: UCSD Pascal - markbessey.blog, accessed June 12, 2026,
<https://markbessey.blog/2025/04/14/a-blast-from-the-past/>
 7. Remembering UCSD P-System, The Pascal Virtual Machine - Hackaday, accessed June 12, 2026,
<https://hackaday.com/2025/04/21/remembering-ucsd-p-system-the-pascal-virtual-machine/>
 8. FOSDEM 2024 - The JVM vs. WebAssembly: An In-Depth Comparative Analysis, accessed June 12, 2026,
<https://archive.fosdem.org/2024/schedule/event/fosdem-2024-2769-the-jvm-vs-webassembly-an-in-depth-comparative-analysis/>
 9. WebAssembly vs. JVM: An Informal Peek - Blog - Erick Guan, accessed June 12, 2026, <https://erickguan.me/2023/wasm-jvm-informal-peek>
 10. What Are the Differences Between WebAssembly and the JVM? - F5, accessed June 12, 2026,
<https://www.f5.com/company/blog/what-are-the-differences-between-webassembly-and-the-jvm>
 11. The JVM vs WebAssembly: An In Depth Comparative Analysis By Shivansh Vij - YouTube, accessed June 12, 2026,
https://www.youtube.com/watch?v=qmVG_JLLq40
 12. Comparing remote procedure calls - National Institute of Standards and Technology, accessed June 12, 2026,
<https://nvlpubs.nist.gov/nistpubs/Legacy/IR/nistir5277.pdf>
 13. ONC RPC concepts - IBM, accessed June 12, 2026,
<https://www.ibm.com/docs/en/cics-ts/6.x?topic=rpc-onc-concepts>
 14. ONC/RPC vs DCE/RPC on HP-UX | PDF | Distributed Computing - Scribd, accessed June 12, 2026,
<https://www.scribd.com/document/449524228/HP-UX-Remote-Procedure-Call-is-it-ONC-RPC-or-DCE-RPC-pdf>
 15. Remote procedure call - Wikipedia, accessed June 12, 2026,
https://en.wikipedia.org/wiki/Remote_procedure_call
 16. Remote Procedure Calls and Web Services - Paul Krzyzanowski, accessed June 12, 2026, <https://pk.org/classes/417/notes/rpc-terms.html>
 17. Network Data Representation - Grokipedia, accessed June 12, 2026,
https://grokipedia.com/page/network_data_representation
 18. Technical Standard DCE 1.1: Remote Procedure Call - The Open Group Publications Catalog, accessed June 12, 2026,
<https://pubs.opengroup.org/onlinepubs/9629399/toc.pdf>
 19. Introduction to DCE, accessed June 12, 2026,
https://www.uvm.edu/~fcs/Doc/DCE/DCE-3.1-Introduction_to_DCE.pdf
 20. Remote Procedure Calls | Linux Journal, accessed June 12, 2026,

- <https://www.linuxjournal.com/article/2204>
21. ONC and DCE - IBM, accessed June 12, 2026,
<https://www.ibm.com/docs/en/cics-ts/6.x?topic=interfaces-onc-dce>
 22. Why MCP Instead of REST? Understanding the Model Context ..., accessed June 12, 2026,
https://medium.com/@ohusiev_6834/why-mcp-instead-of-rest-understanding-the-model-context-protocol-61045b0b4de8
 23. OpenDoc - Wikipedia, accessed June 12, 2026,
<https://en.wikipedia.org/wiki/OpenDoc>
 24. Compound Document | Computer Science | Research Starters - EBSCO, accessed June 12, 2026,
<https://www.ebsco.com/research-starters/computer-science/compound-document>
 25. Why OpenDoc failed, and then failed 3 more times? - Instadeq, accessed June 12, 2026,
<https://instadeq.com/blog/posts/why-opendoc-failed-and-then-failed-3-more-times/>
 26. Why did OpenDoc fail, and then fail 3 more times? - Hacker News, accessed June 12, 2026, <https://news.ycombinator.com/item?id=27106195>
 27. The Decline and Fall of Java on the Desktop Part 1 (1999-2005) - Reddit, accessed June 12, 2026,
https://www.reddit.com/r/java/comments/t56o2u/the_decline_and_fall_of_java_on_the_desktop_part/
 28. IBM System Object Model - Wikipedia, accessed June 12, 2026,
https://en.wikipedia.org/wiki/IBM_System_Object_Model
 29. Fragile binary interface problem - Wikipedia, accessed June 12, 2026,
https://en.wikipedia.org/wiki/Fragile_binary_interface_problem
 30. Why binary compatibility? - c++ - Stack Overflow, accessed June 12, 2026,
<https://stackoverflow.com/questions/15445308/why-binary-compatibility>
 31. The System Object Model (SOM) and the Component Object Model (COM), accessed June 12, 2026,
http://www.cs.cmu.edu/afs/cs/project/gwydion/docs/opendoc/SOM/SOM_vs_COM_Compar.html
 32. 20 ABI (Application Binary Interface) breaking changes every C++ developer should know : r/programming - Reddit, accessed June 12, 2026,
https://www.reddit.com/r/programming/comments/b8v3qs/20_abi_application_binary_interface_breaking/
 33. IBM System Object Model - Encyclopedia.pub, accessed June 12, 2026,
<https://encyclopedia.pub/entry/28114>
 34. Worse is better - Wikipedia, accessed June 12, 2026,
https://en.wikipedia.org/wiki/Worse_is_better
 35. Worse-is-better, accessed June 12, 2026,
<https://cs.stanford.edu/people/eroberts/courses/cs181/projects/2010-11/WorselsBetter/index.php/Worse-is-better.html>
 36. The Rise of "Worse is Better" (original source of the idea... not what I expected

- given usage) : r/programming - Reddit, accessed June 12, 2026,
https://www.reddit.com/r/programming/comments/1z5yku/the_rise_of_worse_is_better_original_source_of/
37. MCP vs. REST: Why do we need a new Protocol ? #1093 - GitHub, accessed June 12, 2026,
<https://github.com/modelcontextprotocol/modelcontextprotocol/discussions/1093>
 38. Model Context Protocol vs. REST API: Which Approach is Better? - ELEKS, accessed June 12, 2026,
<https://eleks.com/expert-opinion/model-context-protocol-rest-api/>
 39. What is Model Context Protocol (MCP)? A guide | Google Cloud, accessed June 12, 2026, <https://cloud.google.com/discover/what-is-model-context-protocol>
 40. Introducing the Model Context Protocol - Anthropic, accessed June 12, 2026,
<https://www.anthropic.com/news/model-context-protocol>
 41. Model Context Protocol (MCP) explained: A practical technical ..., accessed June 12, 2026, <https://codilime.com/blog/model-context-protocol-explained/>
 42. Model Context Protocol (MCP) - Black Hills Information Security, Inc., accessed June 12, 2026, <https://www.blackhillsinfosec.com/model-context-protocol/>
 43. Architecture overview - Model Context Protocol, accessed June 12, 2026,
<https://modelcontextprotocol.io/docs/learn/architecture>
 44. MCP vs. REST: What's the right way to connect AI agents to your API ..., accessed June 12, 2026, <https://workos.com/blog/mcp-vs-rest>
 45. Specification - Model Context Protocol, accessed June 12, 2026,
<https://modelcontextprotocol.io/specification/2025-11-25>
 46. Overview - Model Context Protocol, accessed June 12, 2026,
<https://modelcontextprotocol.io/specification/draft/basic>
 47. Malleable software: Restoring user agency in a world of locked ..., accessed June 12, 2026, <https://www.inkandswitch.com/essay/malleable-software/>
 48. On Embark and Lude - Ink & Switch, accessed June 12, 2026,
<https://www.inkandswitch.com/newsletter/dispatch-001/>
 49. Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model - arXiv, accessed June 12, 2026,
<https://arxiv.org/html/2503.04084v1>